Register map (Smart Serial Controller v2)

All registers are 32 bits wide. Offsets are bytes from the controller's base address:

- **Way 2 (ARM):** base = `0x43C0_0000` (set in `vivado/build_ps_system.tcl`, `SSC_BASEADDR` in `sw/ssc2_regs.h`).
- **Way 1 (FPGA only):** `rtl/top/ssc2_boot.v` uses the offsets directly.

Access types: RO = read only, WO = write only, RW = read/write, W1C = write 1 to clear, W1 = bit is a one-shot command (reads back as 0).

The C names are the ones in `sw/ssc2_regs.h`. The detailed behaviour of each block is in `docs/blocks/`.

range	block	details
`0x000-0x3FF`	v1 registers: UART, SPI, I2C engines, v1 interrupts	section 1
`0x400-0x4FF`	pin crossbar	section 2, `docs/blocks/xbar.md`
`0x500-0x5FF`	network interface configuration	section 3
`0x600-0x6FF`	sensor hub	section 4, `docs/blocks/hub.md`
`0x700-0x7FF`	DMA writer	section 5, `docs/blocks/dma_writer.md`
`0x800-0x9FF`	serial engine	section 6, `docs/blocks/serial_engine.md`
`0xA00-0xAFF`	version, time, second interrupt register, CPU-load counters, board	section 7

Unused addresses read 0 and ignore writes.

---

1. v1 registers (0x000-0x3FF)

These are the v1 registers unchanged, so v1 software still works. Only the ID changed. The v1 bridge engine is replaced by the network, so BRIDGE_CTRL is still a read/write register but does nothing.

1.1 Global

Offset	Name	Access	Bits
`0x000`	ID	RO	`0x53534302` ("SSC", version 2)
`0x004`	INT_STATUS	R / W1C	v1 event flags (table below)
`0x008`	INT_ENABLE	RW	a 1 lets the matching flag drive the IRQ line
`0x00C`	BRIDGE_CTRL	RW	kept for v1 software, no effect in v2
`0x010`	BRIDGE_CNT	RO	reads 0 in v2
`0x014`	SCRATCH	RW	free register for bus tests

INT_STATUS / INT_ENABLE bits (a **level** flag sets itself again while its condition stays true):

Bit	Name	Kind	Meaning
0	UART_RX	level	UART RX FIFO not empty
1	UART_TX_IDLE	level	UART TX FIFO empty and line idle
2	UART_PARITY	event	parity error
3	UART_FRAME	event	framing error (stop bit 0)
4	UART_BREAK	event	line low for a whole frame
5	UART_RX_OVF	event	byte arrived with the RX FIFO full
6	SPI_DONE	event	master: queue ran empty; slave: CS# went high
7	SPI_RX	level	SPI RX FIFO not empty
8	SPI_RX_OVF	event	word arrived with the RX FIFO full
9	I2C_DONE	event	command finished (also after NACK / arbitration loss)
10	I2C_NACK	event	slave did not acknowledge
11	I2C_ARB_LOST	event	another master won the bus
12	I2C_RX	level	I2C RX FIFO not empty
13	UART_TX_OVF	event	write to a full UART TX FIFO
14	SPI_TX_OVF	event	write to a full SPI TX FIFO
15	I2C_TX_OVF	event	write to a full I2C TX FIFO

Status layout shared by UART_STATUS, SPI_STATUS and I2C_STATUS: [0] TX_EMPTY, [1] TX_FULL, [2] RX_EMPTY, [3] RX_FULL, [4] BUSY, [12:8] TX_COUNT, [20:16] RX_COUNT.

1.2 UART (0x100)

Offset	Name	Access	Bits
`0x100`	UART_CTRL	RW	[0] TX_EN, [1] RX_EN, [3:2] data bits (0=5 .. 3=8), [4] PAR_EN, [5] PAR_ODD, [6] STOP2, [7] FLOW, [8] LOOPBACK. W1: [16] TX_FLUSH, [17] RX_FLUSH
`0x104`	UART_BAUD	RW	[15:0] divisor, [19:16] fraction /16; divisor = f_clk / (16 x baud). Reset `0x00040036` = 115200
`0x108`	UART_STATUS	RO	common layout + [5] RX_BUSY, [6] CTS_OK
`0x10C`	UART_TXDATA	WO	[7:0] -> TX FIFO
`0x110`	UART_RXDATA	RO	[7:0] byte from the RX FIFO, [31] EMPTY (nothing was taken)

1.3 SPI (0x200)

Offset	Name	Access	Bits
`0x200`	SPI_CTRL	RW	[0] EN, [1] CPOL, [2] CPHA, [3] LSB_FIRST, [8:4] word length - 1, [10:9] CS_SEL, [11] CS_MANUAL, [12] CS_LEVEL (1 = CS# low), [13] SLAVE, [14] LOOPBACK. W1: [16] TX_FLUSH, [17] RX_FLUSH, [18] ABORT
`0x204`	SPI_CLKDIV	RW	SCLK = 100 MHz / (2 x (DIV + 1)), DIV >= 3. Reset 49 = 1 MHz
`0x208`	SPI_STATUS	RO	common layout
`0x20C`	SPI_TXDATA	WO	word -> TX FIFO (starts a master transfer)
`0x210`	SPI_RXDATA	RO	word from the RX FIFO (0 if empty)

While the SPI network interface runs a transaction it overrides EN, SLAVE, word length (8), CS_MANUAL, CS_LEVEL and CS_SEL (`docs/blocks/ni_spi_i2c.md`).

1.4 I2C (0x300)

Offset	Name	Access	Bits
`0x300`	I2C_CTRL	RW	[0] EN, [1] AUTO_WR. W1: [16] TX_FLUSH, [17] RX_FLUSH, [18] ABORT
`0x304`	I2C_PRESCALE	RW	f_SCL ~= 100 MHz / (4 x (P + 1)). 249 = 100 kHz (reset), 62 = 400 kHz
`0x308`	I2C_ADDR	RW	[9:0] slave address, [15] TEN_BIT
`0x30C`	I2C_CMD	WO	[7:0] LEN, [8] READ, [9] STOP, [10] STOP_ONLY; writing starts it
`0x310`	I2C_STATUS	RO	common layout + [5] HOLDING, [6] NACK, [7] ARB_LOST, [24] BUS_BUSY, [25] SCL, [26] SDA
`0x314`	I2C_TXDATA	WO	[7:0] -> TX FIFO
`0x318`	I2C_RXDATA	RO	[7:0] byte from the RX FIFO, [31] EMPTY

While the I2C network interface runs a transaction it overrides EN, AUTO_WR (0), the address and TEN_BIT (0).

---

2. Pin crossbar (0x400)

Ports: 0 JA, 1 JB, 2 JC, 3 JD, 4 OLED, 5 LED. Sources: 0 OFF, 1 UART, 2 SPI, 3 I2C, 4 SM0, 5 SM1, 6 GPIO, 7 OFF.

Offset	Name	Access	Bits
`0x400 + 4p`	PIN_SEL[p]	RW	**write:** [2:0] new source, [8] FORCE (do not wait for the old source to be idle). **read:** [2:0] current source, [6:4] requested source, [8] switching. Writes while switching are ignored. Reset: JA 1, JB 2, JC 3, JD 0, OLED 0, LED 6
`0x420`	XBAR_STATUS	RO	[5:0] switching, one bit per port
`0x440 + 4p`	SW_CYCLES[p]	RO	clocks from the request to the hand-over of the last switch
`0x460 + 4p`	SW_EDGE[p]	RO	clocks from the request to the first output edge of the new source (0 = none yet)
`0x480`	GPIO_OUT	RW	4 bits per port, port p at [4p+3:4p]
`0x484`	GPIO_OE	RW	4 bits per port (1 = drive)
`0x488`	GPIO_IN	RO	pin levels, 4 bits per port (2-flip-flop synchronised)

Pin roles (pin 0..3 = Pmod pin 1..4):

source	pin 0	pin 1	pin 2	pin 3
UART	CTS# in	TXD out	RXD in	RTS# out
SPI master	CS0# out	MOSI out	MISO in	SCK out
SPI slave	CS# in	MOSI in	MISO out	SCK in
I2C	-	-	SCL open drain	SDA open drain
SM0 / SM1	the SM's pin 0	pin 1	pin 2	pin 3
OLED port	DC	SDIN	RES#	SCLK
LED port	LD0	LD1	LD2	LD3

---

3. Network interface configuration (0x500)

Node ids: 0 serial engine, 1 sensor hub, 2 DMA writer, 3 UART, 4 SPI, 5 I2C.

Offset	Name	Access	Bits
`0x500`	NI_CFG[0] serial engine	RW	[0] EN, [6:4] DEST of SM0's packets, [7] PRIO SM0, [17] WORD4 SM0, [18] WORD4 SM1, [22:20] DEST SM1, [23] PRIO SM1
`0x50C`	NI_CFG[3] UART	RW	[0] EN, [2:1] MODE (0 RAW, 1 FRAME, 2 ADDRESSED), [6:4] DEST, [7] PRIO, [15:8] ARG, [16] RESP_STATUS
`0x510`	NI_CFG[4] SPI	RW	[0] EN
`0x514`	NI_CFG[5] I2C	RW	[0] EN
`0x520 + 4n`	NI_STAT[n]	RO	[15:0] packets that arrived at node n, [31:16] packets node n sent (all six nodes)
`0x540`	NI_UART_TIMEOUT	RW	[15:0] us of silence that ends a RAW packet or drops an incomplete frame. Reset 2000
`0x544`	NI_I2C_TIMEOUT	RW	[15:0] us per I2C command before the NI aborts (status 3). Reset 50000

All NIs are off after reset, so the engines belong to the CPU exactly as in v1. An NI that is off still drains every packet sent to it.

UART frames (MODE 1): `[wlen] [w bytes] [rlen]`, e.g. `01 9F 03` -> XFER_REQ to DEST with ARG (SPI: chip select, I2C: address). MODE 2 adds `[dest] [arg]` in front: `05 4B 01 00 02` reads the TMP2.

---

4. Sensor hub (0x600)

Offset	Name	Access	Bits
`0x600`	HUB_CTRL	RW	[0] EN. W1: [15:8] RUN_NOW, bit 8+t makes task t due now
`0x604`	HUB_STATUS	RO	[0] busy, [3:1] current task, [8] waiting for a reply
`0x608`	HUB_REC_DEST	RW	[2:0] node that gets the RECORDs. Reset 2 (DMA writer)
`0x60C`	HUB_TIMEOUT	RW	reply time-out in ms. Reset 50
`0x610`	HUB_SEQ	RO	sequence number of the next record. Reset 1
`0x614`	HUB_LAST	RO	[3:0] last value length, [15:8] status of the last reply (0xFF = time-out), [23:16] task
`0x618`	HUB_LAST_LO	RO	last value bytes 0..3 (byte 0 in [7:0])
`0x61C`	HUB_LAST_HI	RO	last value bytes 4..7
`0x620`	HUB_CNT_REQ	RO	requests sent
`0x624`	HUB_CNT_RESP_OK	RO	replies with status 0
`0x628`	HUB_CNT_ERR	RO	replies with status != 0
`0x62C`	HUB_CNT_TMO	RO	time-outs
`0x630`	HUB_CNT_REC	RO	records sent
`0x640 + 16t`	TASK[t].W0	RW	[0] EN, [3:1] dest, [6:4] type (1 XFER_REQ, 0 DATA), [7] prio, [15:8] arg, [19:16] wlen, [23:20] rlen, [24] RECORD, [25] SEND_LAST, [26] ADDR_INC, [27] TRIG_RECORDS, [28] APPEND_REC
`0x644 + 16t`	TASK[t].W1	RW	write bytes 0..3 (byte 0 in [7:0])
`0x648 + 16t`	TASK[t].W2	RW	write bytes 4..7
`0x64C + 16t`	TASK[t].W3	RW	[15:0] period (ms, or records with TRIG_RECORDS), [31:16] phase (0 = one period). Writing W3 restarts the countdown

Slide-18 task: `W0 = 0x01214B1B` (EN, node 5, XFER_REQ, 0x4B, wlen 1, rlen 2, RECORD), `W1 = 0`, `W3 = 0x00000064` (every 100 ms).

Record format (16 bytes, little-endian, as stored by the DMA writer): bytes 0-3 timestamp (us), 4 source engine (1 UART, 2 SPI, 3 I2C, 4 SE), 5 data length, 6-7 sequence number, 8-15 data.

---

5. DMA writer (0x700)

Offset	Name	Access	Bits
`0x700`	DMA_CTRL	RW	[0] EN. W1: [8] RESET (ring, indexes and counters to 0)
`0x704`	DMA_BASE	RW	ring start address in DDR (bits 3:0 ignored)
`0x708`	DMA_SIZE_LOG2	RW	[3:0] ring = 2^n records (1..12). Reset 6 = 64 records
`0x70C`	DMA_WR_IDX	RO	next slot the hardware writes
`0x710`	DMA_RD_IDX	RW	next slot software reads; write it to free slots
`0x714`	DMA_BATCH	RW	[7:0] records per batch interrupt. Reset 32. 0 = off
`0x718`	DMA_TIMEOUT	RW	[15:0] ms after the oldest pending record. Reset 1000. 0 = off
`0x71C`	DMA_COUNT	RO	records written
`0x720`	DMA_OVERFLOW	RO	records dropped (ring full)
`0x724`	DMA_AXI_ERR	RO	bursts with a non-OKAY response
`0x728`	DMA_PENDING	RO	records since the last batch event
`0x72C`	DMA_STATUS	RO	[0] busy, [1] full, [2] empty
`0x730`	DMA_IGNORED	RO	packets consumed but not written

The ring is full when `(WR - RD) mod size == size - 1`. Each record is one AXI4 burst of 4 x 32 bits to `BASE + 16 x WR_IDX` through S_AXI_HP0, which bypasses the CPU caches: invalidate the record's cache lines before reading it (`dma_record()` in `sw/ssc2_driver.c`).

---

6. Serial engine (0x800-0x9FF)

Offset	Name	Access	Bits
`0x800`	SE_CTRL	RW	[0] SM0_EN, [1] SM1_EN. W1: [8] SM0_RESTART, [9] SM1_RESTART
`0x804`	SE_FSTAT	RO	[2:0] TX0 level, [6:4] RX0 level, [10:8] TX1 level, [14:12] RX1 level, [16] TX0 full, [17] RX0 empty, [18] TX1 full, [19] RX1 empty, [24] SM0 stalled, [25] SM1 stalled, [28] SM0 RX overflow, [29] SM1 RX overflow (overflows clear on read)
`0x808`	SE_CRC_CFG	RW	[15:0] POLY (reset 0x1021), [31:16] INIT (reset 0xFFFF)
`0x80C`	SE_TIME	RO	microseconds since reset
`0x810` / `0x850`	SM0 / SM1 CLKDIV	RW	[31:16] INT, [15:8] FRAC: one tick every INT + FRAC/256 clocks (INT 0 = 1). Reset 0x00010000
`0x814` / `0x854`	PINCTRL	RW	[1:0] OUT_BASE, [3:2] SET_BASE, [6:4] SET_COUNT (0 = 4), [9:8] IN_BASE, [11:10] SIDE_PIN, [12] SIDE_EN, [14:13] JMP_PIN, [19:16] OD_MASK, [23:20] INIT_OUT, [27:24] INIT_OE
`0x818` / `0x858`	SHIFTCTRL	RW	[0] OUT_SHIFT_LEFT, [1] IN_SHIFT_LEFT, [12:8] START_PC
`0x81C` / `0x85C`	TXF	WO	push a word into the TX FIFO (dropped if full)
`0x820` / `0x860`	RXF	RO	pop a word from the RX FIFO (0 if empty)
`0x824` / `0x864`	EXEC	WO	[15:0] run this instruction at the next tick
`0x828` / `0x868`	STATE	RO	[4:0] PC, [8] stalled, [9] enabled, [10] EXEC pending
`0x82C` / `0x86C`	X	RO	X register
`0x830` / `0x870`	Y	RO	Y register
`0x834` / `0x874`	CRC	RO	[15:0] CRC-16 register
`0x838` / `0x878`	PINS	RO	[3:0] in, [7:4] out, [11:8] oe
`0x900 + 4i`	PROG[i]	RW	[15:0] instruction i, i = 0..31

The instruction set and the program word formats are in `docs/blocks/serial_engine.md` and `docs/blocks/se_programs.md`.

---

7. Misc and second interrupt register (0xA00)

Offset	Name	Access	Bits
`0xA00`	VERSION	RO	`0x00020000`
`0xA04`	TIME_US	RO	microseconds since reset (wraps after 71 minutes)
`0xA08`	INT2_STATUS	R / W1C	v2 event flags (table below)
`0xA0C`	INT2_ENABLE	RW	a 1 lets the matching flag drive the IRQ line
`0xA10`	APB_COUNT	RO, write clears	number of register accesses (APB transfers)
`0xA14`	IRQ_COUNT	RO, write clears	rising edges of the IRQ line
`0xA18`	OLED_PWR	RW	[0] OLED VDD pin, [1] OLED VBAT pin; 1 = supply off. Reset 3
`0xA1C`	BOARD_IN	RO	[7:0] switches SW7..0, [12:8] buttons C, D, L, R, U
`0xA20`	SE_LOOP	RW	[0] SM1 pin 2 listens to SM0 pin 1 inside the chip

INT2_STATUS / INT2_ENABLE bits:

Bit	Name	Kind	Meaning
0	DMA_BATCH	event	a batch of records is ready (BATCH reached or time-out)
1	DMA_OVERFLOW	event	a record was dropped (ring full)
2	DMA_AXI_ERR	event	a burst got an error response
3	HUB_ERROR	event	a hub request failed (error status or time-out)
4	HUB_RECORD	event	the hub made a record
5	XBAR_SWITCH	event	a crossbar port handed over to a new source
6	SE_RX0	level	SM0's RX FIFO is not empty
7	SE_RX1	level	SM1's RX FIFO is not empty

The IRQ line (to IRQ_F2P, interrupt 61) is `(INT_STATUS & INT_ENABLE) != 0` OR `(INT2_STATUS & INT2_ENABLE) != 0`.

Measuring CPU load (slide 24, measurement 4): write IRQ_COUNT, then APB_COUNT (the clearing write itself is not counted), let the work run, then read both. `tb_ssc2_system` measures 0 accesses and 1 interrupt for 32 hub readings, against 10 accesses and 2 interrupts for one CPU-driven (v1-style) reading.